De mirar a reservar

Rediseño de RoomsScreen: encontrar una habitación que le sirva al huésped y reservarla sin equivocarse de número.

Rony Valdelamar
Grupo 4

Jair Sánchez Pérez
Grupo 4

Victor Rocha Ramos
Grupo 4

Aníbal Meza Anaya
Grupo 4

¿Para qué sirve RoomsScreen?

Es la ventana donde el recepcionista consulta qué habitaciones están libres antes de ofrecerle una al huésped.

REQUISITO

RF03

Consultar las habitaciones disponibles, agrupadas por tipo: Sencilla, Doble y Suite.

USUARIO

Recepcionista

Atiende al huésped en el mostrador y necesita decidir rápido qué habitación le sirve.

TAREA

Elegir y reservar

Encontrar una habitación con la capacidad correcta y reservarla sin equivocarse de número.

Login → Menú → **Consultar habitaciones** → Reservar → Check-in → Check-out

Así era antes

Funcionaba bien, pero **solo servía para mirar**: mostraba la información y dejaba todo el trabajo al recepcionista.

1. **Chip que no informa**
   Siempre dice "Disponible", porque solo se listan disponibles.
2. **Filas sin acción**
   No se puede reservar desde aquí.
3. **Capacidad escondida**
   Hay que leer fila por fila para saber cuál sirve.
5. **Sin forma de actualizar**
   Es `StatelessWidget` : no hay botón ni gesto para refrescar.



Vista recreada a partir del código original (rooms_screen.dart, 93 líneas)

Seis oportunidades de mejora

P1 · CONSISTENCIA

Chip redundante

Siempre dice "Disponible" en verde: ocupa espacio sin dar información.

`_EstadoChip` · líneas 75–92

P2 · INTERACCIÓN

La lista no lleva a ninguna acción

Para reservar hay que volver al menú y escribir el número de memoria.

`ListTile` sin `onTap` · líneas 53–59

P3 · ORGANIZACIÓN

No ayuda a elegir

Con un grupo de 3, el recepcionista debe deducir que solo sirve una Suite.

Capacidad solo en el `subtitle` · línea 57

P4 · JERARQUÍA

Sin panorama general

Con "0 disponibles" no se sabe si el hotel está lleno o qué está ocupado.

Solo `consultarDisponiblesAgrupadas()`

P5 · RETROALIMENTACIÓN

Datos que se quedan viejos

No hay forma de actualizar la lista sin salir y volver a entrar.

`StatelessWidget` · línea 14

P6 · LEGIBILIDAD

Textos y estado vacío

"disponible(s)", "persona(s)"; el aviso vacío no sugiere qué hacer.

Líneas 42 y 49

Criterios de la guía: jerarquía, organización, legibilidad, consistencia, interacción, retroalimentación

Cinco mejoras, en orden

	MEJORA	RESUELVE	BENEFICIO ESPERADO	CÓMO SE COMPRUEBA
01	**Botón "Reservar"** con el número precargado	P2	Menos pasos y cero errores al digitar el número	Reservar desde la lista sin escribir el número
02	**Filtro "¿Cuántas personas?"**	P3	Ver solo las habitaciones que le sirven al huésped	Elegir "3–4" y ver que solo quedan Suites
03	**Resumen superior** de estados	P4	Panorama del hotel de un vistazo	Reservar y ver que los contadores cambian
04	**Actualización** automática y manual	P5	La lista siempre refleja el estado real	La habitación reservada desaparece al volver
05	**Limpieza visual y de textos**	P1 · P6	Pantalla más limpia y fácil de leer	Comparar antes/después; ventana de 360 px

Prioridad = efecto directo sobre la tarea del usuario, no lo decorativo

El boceto

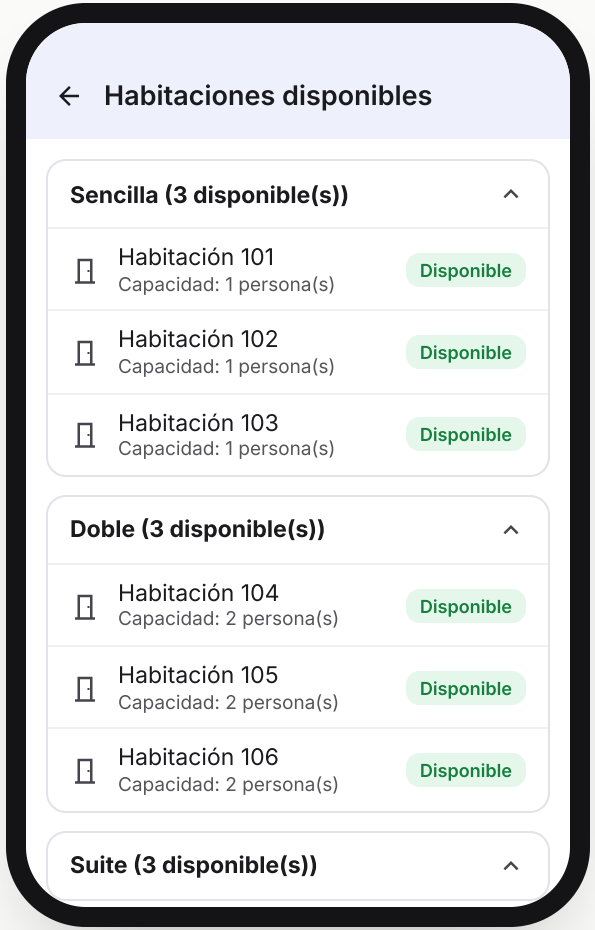


- **Jerarquía:** primero el contexto (resumen), luego el filtro y al final la lista.
- **Acción principal:** botón `FilledButton.tonal` a la derecha de cada fila; tonal para no competir con "Confirmar reserva".
- **Estados:** el filtro elegido queda resaltado y la lista se redibuja al volver de reservar.
- **Estado vacío:** mensaje con sugerencia ("Ajusta el filtro...").
- **Adaptabilidad:** `Row` + `Expanded` en el resumen y `ListView` con `padding`, para no desbordar en pantallas angostas.

Cada bloque del boceto responde a un problema del diagnóstico

Antes y después

ANTES



DESPUÉS



1. **Reservar desde la fila**
   Abre la reserva con el número ya escrito.
2. **Filtro por personas**
   Muestra solo lo que le sirve al grupo.
3. **Resumen del hotel**
   Disponibles, reservadas y ocupadas.
4. **Actualizar**
   Botón en la barra + gesto de arrastrar.
5. **Textos limpios**
   Sin chip redundante; "3 disponibles", "1 persona".

Vistas recreadas a partir del código de cada versión; en la sustentación se muestran en vivo

De Stateless a Stateful

ANTES · LÍNEA 14

```
class RoomsScreen
    extends StatelessWidget {
  final Hotel hotel;
  // sin estado propio
}
```

DESPUÉS

```
class RoomsScreen
    extends StatefulWidget { ... }

class _RoomsScreenState ... {
  int _filtroSeleccionado = 0;
}
```

POR QUÉ

Guardar el filtro

La pantalla ahora recuerda qué opción eligió el recepcionista.

SETSTATE()

Redibujar

Al cambiar el filtro, al volver de reservar y al pulsar "Actualizar".

HOTEL

Solo se lee

Llega por el constructor como antes: `widget.hotel`.

Reservar desde la lista

ROOMS_SCREEN.DART

```dart
Future<void> _irAReservar(Habitacion h) async {
  await Navigator.of(context).push(
    MaterialPageRoute(
      builder: (_) => ReserveScreen(
        hotel: widget.hotel,
        numeroInicial: h.numero,
      ),
    ),
  );
  if (mounted) setState(() {});
}
```

RESERVE_SCREEN.DART · ÚNICO CAMBIO FUERA

```dart
final int? numeroInicial;

void initState() {
  super.initState();
  if (widget.numeroInicial != null) {
    _numeroHabitacionController.text =
        '${widget.numeroInicial}';
  }
}
```

WIDGET

FilledButton.tonal

En el `trailing` de cada `ListTile`.

AWAIT

Espera el regreso

Cuando el usuario vuelve, `setState` redibuja y la habitación reservada ya no aparece.

OPCIONAL

Sin romper nada

Si no se envía `numeroInicial`, ReserveScreen funciona igual que antes.

Filtro, resumen y actualizar

FILTRO · P3

```
_FiltroPersonas('Todas', 1),
_FiltroPersonas('1', 1),
_FiltroPersonas('2', 2),
_FiltroPersonas('3–4', 3),

bool _cumpleFiltro(h, minimo)
  => h.capacidad >= minimo;

// Wrap de ChoiceChip
onSelected: (_) => setState(
  () => _filtroSeleccionado = i),
```

RESUMEN · P4

```
disponibles = hotel.habitaciones
  .where((h) => h.estaDisponible)
  .length;
reservadas = ...estaReservada
ocupadas   = ...estaOcupada

// Row con 3 Expanded
Row(children: [
  Expanded(child: _TarjetaResumen(..)),
  ...
])
```

ACTUALIZAR · P5

```
actions: [
  IconButton(
    tooltip: 'Actualizar',
    icon: Icon(Icons.refresh),
    onPressed: _refrescar,
  ),
],
body: RefreshIndicator(
  onRefresh: _refrescar,
  child: ListView(physics:
   AlwaysScrollableScrollPhysics(),
```

El gesto de arrastrar no funciona con mouse en Chrome; por eso también hay botón.

Hotel no se tocó

- `lib/services/hotel.dart` quedó **idéntico** al original.
- La disponibilidad se sigue validando en `reservarHabitacion()`.
- La capacidad se sigue validando en `realizarCheckIn()`.
- El filtro y el resumen **solo leen** datos; no deciden nada.

```
RoomsScreen
├─ lee   → hotel.habitaciones
├─ lee   → consultarDisponiblesAgrupadas()
└─ abre  → ReserveScreen(numeroInicial: 104)
            └─ llama → hotel.reservarHabitacion()
                       valida: existe, disponible, días > 0

La interfaz presenta y delega.
Las reglas viven en un solo lugar.
```

Qué vamos a mostrar

01 · ABRIR

Consultar habitaciones

Se ve el resumen **9 · 0 · 0** y el filtro en "Todas".

02 · FILTRAR

Elegir "2"

Las Sencillas quedan vacías; solo aparecen Dobles y Suites.

03 · RESERVAR

Pulsar "Reservar" en la 104

ReserveScreen abre con **104** ya escrito.

04 · VOLVER

Confirmar y regresar

La 104 desaparece y el resumen pasa a **8 · 1 · 0**.

05 · ERROR DE DATOS

Poner 0 días

El campo muestra "Debe ser mayor que cero" y no se reserva.

06 · ERROR DE NEGOCIO

Reservar otra vez la 104

Hotel responde: "La habitación 104 no está disponible."

Además: ventana angosta (~360 px) para revisar que nada se desborde

Lo que comprobamos

	PRUEBA AUTOMATIZADA (FLUTTER TEST)	RESULTADO
A1	Aparecen el resumen, el filtro y 9 botones "Reservar"	Comprobado
A2	"3–4" deja solo las 3 Suites	Comprobado
A3	"Reservar" abre ReserveScreen con "101"	Comprobado
A4	Estado vacío con sugerencia	Comprobado
A5	"Actualizar" refleja una reserva hecha en Hotel	Por confirmar

	PRUEBA MANUAL (CHROME)	RESULTADO
M1	Uso normal: reservar la 104	En la demo
M2	Datos incorrectos: 0 días	En la demo
M3	Error de negocio: doble reserva	En la demo
M5	Pantalla angosta ~360 px	En la demo
M6	Botón "Actualizar"	En la demo

Todavía es hipótesis: que el filtro reduce el tiempo de elección. No se ha probado con usuarios, así que se presenta como beneficio **esperado**.

Registro completo en evidencias/PRUEBAS.md

Problema → cambio → evidencia

PROBLEMA INICIAL	CAMBIO IMPLEMENTADO	EVIDENCIA	CONCLUSIÓN
P2 · sin acción	Botón "Reservar" + `numeroInicial`	Prueba A3 + código	Comprobado
P3 · no ayuda a elegir	Filtro con `ChoiceChip`	Prueba A2	Comprobado
P4 · sin panorama	Resumen de 3 estados	Prueba A1	Comprobado
P5 · datos viejos	`StatefulWidget` + Actualizar + `RefreshIndicator`	Código; A5 y M1 por confirmar	Observado
P6 · textos y vacío	Plurales y estado vacío con sugerencia	Prueba A4	Comprobado
P1 · chip redundante	Se quitó `_EstadoChip`; capacidad con ícono	Comparación antes/después	Observado
Elegir más rápido	Filtro + resumen	Sin prueba con usuarios	Esperado

Comprobado: una prueba lo demuestra · Observado: cambio visible en código/pantalla · Esperado: falta probarlo

¿Por qué es significativa?

EL CAMBIO CLAVE

De mirar a actuar

El problema principal era P2: la pantalla no llevaba a ninguna acción. Ahora se reserva desde la fila, sin escribir el número.

LO QUE APRENDIMOS

UX sin tocar reglas

Separar interfaz y lógica permitió cambiar la experiencia sin modificar `Hotel` ni arriesgar las validaciones.

PRINCIPIO APLICADO

Menos pasos, menos errores

Prevenir el error (número precargado) vale más que decorar la pantalla.

SIGUIENTE VERSIÓN

Ver todos los estados

Mostrar también reservadas y ocupadas, reutilizando el chip de color.

Probar con usuarios

Medir si el filtro reduce el tiempo de elección.

Guardar los datos

Hoy todo vive en memoria: al recargar se pierde.

Lo que nos pueden preguntar

01 **¿Cuál era el problema más importante?**
Que la lista no llevaba a ninguna acción: para reservar había que volver al menú y escribir el número de memoria.

02 **¿Por qué esta solución y no otra?**
Precargar el número elimina el paso que causaba errores, sin tocar `Hotel` ni crear pantallas nuevas.

03 **¿Qué principio de diseño aplicaron?**
Reducir pasos y prevenir errores; jerarquía: contexto, luego filtro, luego acción.

04 **¿Qué widget hizo posible el cambio?**
`FilledButton.tonal` + `Navigator.push` con `numeroInicial`; `ChoiceChip` para el filtro; `setState` para redibujar.

05 **¿Qué conservaron y por qué?**
Toda la lógica en `Hotel`: la interfaz presenta y delega, y las reglas no se duplican.

06 **¿Cómo comprobaron que funciona?**
Pruebas de widget con `flutter test` y la demostración manual en Chrome.

07 **¿Qué sigue siendo hipótesis?**
Que el filtro hace elegir más rápido: no se ha probado con usuarios.